AutoIA: Roadmap from 0.21.0.dev1 to Version 1.0

Release Blockers, Validation Gates, Performance Milestones, and Deferred Research

August 2026

Abstract

AutoIA 0.21.0.dev1 already contains the central mathematical architecture needed for an initial validated-execution release: machine-rigorous binary64 proof kernels, polynomial and shared-generator representations, rigorously certified smooth elementary functions and normal-distribution functions, semantic parameter-dependent linear solves, path-aware scalar control flow, practical subdivision, fast rigorous matrix reductions, and prepared repeated-query execution with certificate restriction and multicore CPU batching.

The remaining question is no longer "what else can AutoIA possibly support?". The correct Version-1 question is: "what must be hardened, completed, measured, and frozen so an outside engineer can install the package, understand its supported semantics, obtain trustworthy results, and diagnose a failed proof?" This roadmap therefore separates release blockers from useful follow-on research. It intentionally avoids making GPU proof execution, a large semantic-primitive zoo, or every NumPy operation prerequisites for Version 1.0.

Contents

1 Current baseline: what is already complete **3**

2 Definition of Version 1.0 **3**

3 Priority classes **4**

4 P0 blocker 1: resolve the centered-domain boundary issue cleanly **4**
4.1 Recommended solution direction 5
4.2 Acceptance tests 5

5 P0 blocker 2: batch-native polynomial/range performance (Step 9d) **5**
5.1 Required work 5
5.2 Release acceptance targets 6

6 P0 blocker 3: external-platform proof validation and CI **6**

7 P0 blocker 4: external engineering validation corpus **6**
7.1 Recommended corpus 7
7.2 What to measure 7

8 P0 blocker 5: freeze the Version-1 supported-language contract **7**

9 P0 blocker 6: API, packaging, and release engineering **8**

10 P0 blocker 7: documentation and second-developer reproducibility **9**

11 P1: improve prepared many-box scaling beyond the current bounded scan **9**

12 P1: numeric batch-buffer architecture **10**

13 P1: generator growth and sound compression **10**

14 P1: broader solve-family reuse **10**

15 GPU roadmap: not a Version-1 proof blocker **11**
15.1 Stage G1: GPU proposal backend 11
15.2 Stage G2: experimental directed-rounding proof kernels 11
15.3 Stage G3: optional trusted GPU release 11

16 P2/post-1.0 semantic expansions **11**

17 Suggested milestone sequence **12**

18 Quantitative release gates **12**

19 What should deliberately not be done before Version 1 **13**

20 Immediate next action **13**

21 Final release perspective **13**

1 Current baseline: what is already complete

The current checkpoint is AutoIA `0.21.0.dev1`. The final Step-9c tree passes 223 regression/unit tests, builds an installable wheel, and passes a clean wheel smoke test including generator-valued linear solve, exact/restricted certificate cache reuse, and many-box bounds output.

The following major capabilities are already implemented:

1. **Architecture and public failure taxonomy.** Trusted numerical, polynomial, enclosure, semantic, controller, and public API layers are separated; proof/runtime policy state is context-local rather than process-global.

2. **Machine-rigorous binary64 reference backend.** Environment audit, outward scalar arithmetic, exact-rational regression, vectorized outward proof reductions, and fail-closed numerical limits.

3. **Structured scalar representations.** Interval, polynomial plus remainder, shared-generator polynomial, and piecewise/path results.

4. **Smooth scalar functions.** Validated `sqrt`, `exp`, `log`, `sin`, `cos`, constant real powers, `erf`, normal PDF/CDF, plus common SciPy syntax compatibility.

5. **Ordinary NumPy engineering arrays.** Object-array construction, indexing/slicing, broadcasting, common reductions, dot/matmul, Euclidean/Frobenius norms, and output shape retention for the documented subset.

6. **General semantic linear solve.** Higher-order parameterized matrix families, residual/contraction certification, preconditioner portfolio, multiple right-hand sides, and generator-aware solve predictors.

7. **Uncertain scalar control flow.** Path-aware `if`/`elif` execution, exact simple boundaries, subdivision plus conservative path enumeration, and fail-closed resource budgets.

8. **Practical adaptive subdivision.** Cheap structural split score with a realized-gain safeguard; expensive lookahead retained as a research oracle.

9. **Performance improvements.** Rigorous vectorized/batched reductions, proposal factorization reuse, sparse polynomial caches, and strong end-to-end semantic-solve speedups without trusting BLAS as a proof oracle.

10. **Prepared repeated-query architecture.** `prepare(model,...)`, exact cache, rigorous interior certificate restriction, serial/thread/process many-box execution, and streaming bounds interfaces.

This is enough functionality to justify moving toward an external release rather than continuing an open-ended feature expansion.

2 Definition of Version 1.0

Version 1.0 should mean:

An outside user can install AutoIA on a supported platform, pass a documented subset of ordinary engineering Python/NumPy code and interval input boxes, and receive either (i) a machine-rigorous output certificate with documented semantics or (ii) a typed fail-closed explanation. The package has a frozen public API, a reproducible independent validation suite, realistic performance on representative engineering models, and enough documentation that a second developer can safely extend the code without relying on undocumented project history.

Version 1.0 does *not* need to mean:

- every NumPy/SciPy operation is supported;
- exact interval hulls are always obtained;
- every failed proof can be rescued automatically;
- GPU proof execution is available;
- arbitrary ODE, nonlinear root, eigensolver, or optimization libraries are semantic primitives;
- all correlated physical input domains are represented intrinsically.

The release should be narrow enough to be defensible and broad enough to be useful.

3 Priority classes

This roadmap uses four priority classes.

Priority	Meaning
P0	Required before Version 1.0 because correctness, reproducibility, or basic external usability is not yet sufficiently mature without it.
P1	Strongly desirable before Version 1.0, but may be deferred if it threatens release focus and the limitation can be clearly documented/fail-closed.
P2	Natural post-1.0 capability/performance work enabled by the frozen architecture.
Research	Longer-term methodological extensions that should not delay the first stable release.

4 P0 blocker 1: resolve the centered-domain boundary issue cleanly

The current polynomial input representation constructs a rigorous symmetric centered domain that can extend slightly beyond the original user interval due to outward rounding. This is sound, but it can cause a valid physical domain such as

$$x \in [0, 4]$$

to produce a represented polynomial argument range with a tiny negative lower endpoint, causing $\sqrt{x}$ or $\log(x)$ to fail closed at an exact boundary.

This issue should be addressed before Version 1.0 because it is likely to confuse outside users and because ad hoc clipping would be mathematically unacceptable.

4.1 Recommended solution direction

Introduce explicit physical-domain information distinct from the symmetric polynomial coordinate box. A validated scalar should be able to know both:

1. the symmetric domain used to bound polynomial monomials; and
2. the exact/original physical interval information that may prove operation specific domain constraints.

For an operation such as $\sqrt{x}$, AutoIA should prove nonnegativity of the actual represented semantic quantity/domain rather than infer it by arbitrarily clipping a polynomial range artifact.

4.2 Acceptance tests

P0 is complete when:

- exact-boundary $\sqrt{[0, b]}$ and appropriate log boundary cases have documented rigorous behavior;
- the solution does not silently narrow a proof interval;
- interior cases remain no-regression;
- prepared certificate restriction respects the richer domain metadata;
- boundary behavior is tested under both x86-64 and ARM64 supported platforms.

5 P0 blocker 2: batch-native polynomial/range performance (Step 9d)

Step 9b moved the bottleneck away from rigorous matrix reductions. Profiling now identifies polynomial coefficient ranging and monotonicity-reduced Bernstein transforms as dominant costs. Step 9c also froze a many-box public API before optimizing these internals.

The next performance implementation should therefore be *batch-native*, not an isolated one-box micro-optimization.

5.1 Required work

1. Profile `range_interval_coefficient_poly` and Bernstein conversion on representative solve and non-solve models.
2. Replace repeated Python-object coefficient loops where possible by numeric structure-of-arrays buffers.
3. Vectorize/batch monomial and coefficient transforms across outputs and, when available, across many input boxes.
4. Preserve a slow scalar/reference path for regression and exact audit.
5. Revisit the Bernstein transform algorithmically; the earlier simple precomputation attempt was measured and correctly reverted because it was slower.
6. Ensure any changed arithmetic order remains rigorously outward and covered by independent exact/high-precision tests.

5.2 Release acceptance targets

No single speed number should be a universal requirement, but Version 1 should have:

- an end-to-end benchmark suite with fixed representative models;
- no major proof kernel dominated by avoidable Python loops at moderate array sizes;
- demonstrated useful scaling from one box to hundreds/thousands of boxes;
- documented performance guidance for polynomial order, process workers, cache use, and subdivision.

6 P0 blocker 3: external-platform proof validation and CI

The trusted kernel is audited at runtime, but Version 1 requires repeated release validation on the platforms users are expected to run.

At minimum, establish continuous integration or reproducible release jobs for:

- Linux x86-64;
- macOS ARM64, important for current Apple Silicon engineering users;
- preferably Windows x86-64 if Windows will be claimed as a supported Version-1 platform.

Each supported platform should run:

1. trusted environment audit;
2. exact-rational scalar arithmetic tests;
3. exact-rational reduction tests, including vectorized paths;
4. high-precision elementary/special-function tests;
5. near-singular semantic solve tests;
6. branch/path tests;
7. prepared certificate restriction tests;
8. wheel installation smoke tests.

If a platform cannot satisfy the trusted binary64 assumptions, AutoIA should explicitly refuse machine-rigorous execution there rather than weaken the audit.

7 P0 blocker 4: external engineering validation corpus

The current test suite is strong internally but heavily developed by the same research effort that created the algorithms. Version 1 should include models written or selected independently of AutoIA implementation decisions.

7.1 Recommended corpus

Include at least:

- mechanical spring/structural systems;
- small thermal/fluid algebraic models;
- parameterized linear systems with nonlinear coefficients;
- models using normal CDF/PDF in reliability calculations;
- branching engineering logic;
- vector/matrix assembly and reductions;
- repeated-query workloads with overlapping and unrelated interval boxes;
- deliberately difficult near-singular and domain-boundary cases.

At least some models should come from people who did not participate in the core AutoIA coding effort.

7.2 What to measure

For each model record:

- whether unmodified user syntax is supported;
- certificate success/failure and typed reason;
- output widths and comparison to dense/high-precision sampling when feasible;
- runtime and memory;
- sensitivity to polynomial order and representation;
- subdivision/path count;
- repeated-query cache hit/restriction rate and multicore scaling.

Sampling is diagnostic, not the proof; its purpose is to identify unexpected width inflation or implementation mistakes.

8 P0 blocker 5: freeze the Version-1 supported-language contract

The current subset is documented, but external testing will inevitably reveal common idioms that are missing. Before Version 1, make an explicit decision for each common category: implement it or document a fail-closed exclusion.

Particular candidates to evaluate are:

- scalar `abs`/`np.abs`;
- scalar and array `minimum`/`maximum` and `clip`;

- `np.where` with uncertain scalar/elementwise conditions;
- common helper functions imported from neighboring Python modules;
- local imports inside model functions;
- common `einsum`/`tensordot` cases;
- additional norms used in engineering code.

It is not necessary to implement all of these. The release blocker is ambiguity. A user should be able to determine from the manual whether a construct is supported, and unsupported validated values must fail closed rather than be silently coerced.

A short corpus study should drive this decision rather than adding operations because they are easy in isolation.

9 P0 blocker 6: API, packaging, and release engineering

Before 1.0, freeze and test the public surface exposed from `autoia/__init__.py`. This includes:

- `autoia`;
- `autoia_subdivide`;
- `autoia_adaptive_subdivide`;
- `prepare` and `PreparedAutoIA`;
- interval/backend/policy/result classes;
- special normal/error-function helpers;
- public exception classes.

Required release-engineering tasks include:

1. choose an actual software license; the current placeholder is not suitable for outside release;
2. complete package metadata, project URLs, authorship/maintainer information, and Python classifiers;
3. define supported Python/NumPy/SciPy version ranges and test them;
4. build wheels/sdist reproducibly;
5. add clean-environment installation tests;
6. ensure tests, historical oracle, and audit-only dependencies are not shipped as runtime package code unless deliberately included;
7. add a concise public guarantee statement and supported-language reference;
8. add changelog/release notes and semantic-versioning rules.

10 P0 blocker 7: documentation and second-developer reproducibility

The Version-1 documentation set should include, at minimum:

1. a self-contained mathematical specification;
2. a theory-to-source implementation map;
3. a user/developer software manual;
4. the release roadmap and explicit deferred scope;
5. installation and several copyable engineering examples;
6. typed-error troubleshooting guidance;
7. instructions for running the independent audits and release tests.

A practical acceptance criterion is that a technically capable second developer who did not participate in the previous chat/research history can identify where to add a new elementary function or semantic primitive and can explain which calculations are proposals versus proof-critical calculations.

11 P1: improve prepared many-box scaling beyond the current bounded scan

Step 9c deliberately uses a simple bounded LRU cache and linear containment scan. This is appropriate for freezing semantics, but a million-query workload may need more sophisticated indexing.

Strong pre-1.0 improvements, if time permits, are:

- spatial indexing of cached physical boxes;
- persistent certificate stores for long-running jobs;
- better policy for deciding when to certify a larger parent/master box versus many individual children;
- sharp restriction of pathwise/piecewise cached certificates;
- metrics for cache coverage, reuse depth, and restriction cost.

These are P1 rather than P0 because the current behavior is already sound and usable. Version 1 can document that process-parallel fresh proofs are the fallback for unrelated boxes.

12 P1: numeric batch-buffer architecture

The current tracing representation uses NumPy object arrays containing Python validated scalar objects. This is excellent for compatibility with ordinary engineering code but not optimal for very large batched workloads or GPU execution.

A desirable pre-1.0 internal refactor is a numeric structure-of-arrays buffer for hot polynomial/range paths, with dimensions conceptually like

$$(N,\ n_{\text{terms}},\ n_{\text{outputs}}),$$

where N is the number of simultaneously processed boxes.

This should remain an internal backend detail behind `PreparedAutoIA`; the public API should not force users to know the buffer format. If the work is too large for the release schedule, the API can be frozen in 1.0 and the buffer backend added afterward without breaking users.

13 P1: generator growth and sound compression

The shared-generator representation preserves useful provenance but can accumulate many auxiliary IDs. A naive merge can invent false correlation or destroy important cancellation.

Any generator compression must therefore have an explicit inclusion contract. Possible strategies include:

- preserve a selected set of high-impact shared generators and convert the rest to independent box error;
- group generators only through a mathematically justified outer approximation;
- cap generator count with a documented monotone loss of correlation.

This is strongly desirable for large vector models, but not a correctness blocker for Version 1 if the uncompressed mode remains usable and documented.

14 P1: broader solve-family reuse

Prepared repeated-query workloads may solve closely related matrix families many times. Additional safe proposal reuse could include:

- previous-box preconditioners;
- previous predictor coefficients;
- reference matrix/factorization reuse when the exact point matrix is unchanged;
- shared matrix-family bounds across multiple right-hand sides;
- warm starts across neighboring boxes.

Because these quantities are proposals, aggressive reuse is mathematically safe provided each new box independently passes the residual/contraction certificate. This is an attractive performance follow-on after Step 9d profiling.

15 GPU roadmap: not a Version-1 proof blocker

GPU capability should be pursued, but a full trusted GPU backend should not delay Version 1.0.

15.1 Stage G1: GPU proposal backend

After numeric batch buffers exist, use a GPU for non-proof operations such as:

- batched nominal/reference linear solves;
- candidate preconditioners;
- polynomial predictor coefficient generation;
- controller estimates;
- other large proposal-only matrix operations.

The CPU trusted backend then certifies the actual stored GPU proposal. This gives useful acceleration with minimal change to the trust boundary.

15.2 Stage G2: experimental directed-rounding proof kernels

A proof-producing GPU backend requires explicit control of directed rounding, subnormal/flush-to-zero behavior, reduction ordering, and independent exact validation. Ordinary GPU NumPy analogues must not be assumed to reproduce the CPU trusted semantics.

15.3 Stage G3: optional trusted GPU release

Only after cross-device audits and adversarial tests should GPU-produced bounds be allowed to set `machine_rigorous=True`. Until then, GPU should be viewed as a proposal accelerator.

16 P2/post-1.0 semantic expansions

Once the Version-1 architecture is stable, natural next semantic primitives include:

- nonlinear root systems using interval Newton/Krawczyk-style certificates;
- eigensystem/singular-value bounds;
- ODE/DAE integration with validated flow enclosures;
- optimization semantic primitives;
- additional probability distributions/special functions;
- common sparse linear algebra;
- richer non-box physical input domains.

These should be added only when there is an explicit mathematical semantics and independent proof mechanism. Their absence should not hold the first release hostage.

17 Suggested milestone sequence

The exact version numbers are less important than the gates, but the following sequence is recommended.

Milestone	Required outcome
M1: Step 9d	Batch-compatible polynomial coefficient/range/Bernstein optimization; profile after changes; preserve scalar oracle.
M2: Domain hardening	Resolve centered-domain boundary semantics without clipping; add boundary regression on supported platforms.
M3: External corpus	Run independent engineering models; decide the final common-language subset from actual usage; implement only high-value missing idioms.
M4: Platform/release hardening	CI on supported architectures, wheel/sdist pipeline, license/metadata, clean installation audits, version/dependency matrix.
M5: Release candidate	Freeze public API and guarantee statement; documentation review by a second developer/user; no new feature categories unless they fix a release blocker.
M6: Version 1.0	Publish stable package with known limitations, benchmark/audit artifacts, and a post-1.0 research roadmap.

18 Quantitative release gates

The project benefits from explicit go/no-go gates rather than a vague sense of completeness. Suggested gates are:

1. **Regression gate:** all frozen tests pass on every supported release platform.
2. **Trusted audit gate:** zero containment failures in exact-rational scalar/reduction stress suites and independent high-precision elementary suites.
3. **External corpus gate:** all unsupported constructs fail with a typed message; all returned certificates contain independent truth samples/reference solutions; no silent numeric coercion is observed.
4. **Boundary gate:** documented $\sqrt{\cdot}$/log physical-domain boundary cases behave rigorously and predictably.
5. **Many-box gate:** prepared streaming can process a large synthetic stream without unbounded memory growth; process mode scales materially on a multicore machine; repeated/contained query reuse demonstrably avoids model execution.
6. **Packaging gate:** clean wheel installation and smoke tests pass in fresh environments for every supported Python version/platform.
7. **Documentation gate:** a second developer can build, test, modify, and explain the proof boundary using only repository documentation.

A million-box benchmark should be included before release, but the target should be framed as throughput/memory/scaling characterization rather than an arbitrary absolute runtime that depends strongly on model complexity and hardware.

19 What should deliberately not be done before Version 1

To prevent scope drift, the following should not become mandatory pre-1.0 work unless external validation reveals a critical use case:

- full rigorous GPU certification;
- automatic recognition of arbitrary hand-coded linear/nonlinear algorithms;
- a large zoo of alternative enclosure representations;
- arbitrary correlated stochastic input objects;
- full Python control-flow semantics;
- general compiler/AST transformation of every helper module;
- support for every SciPy distribution or special function;
- global optimization or ODE certification merely because they are attractive research topics.

The first release should establish credibility of the architecture and proof contract. New semantic domains can then build on that stable foundation.

20 Immediate next action

With Step 9c complete, the recommended immediate engineering action is:

> Implement Step 9d by restructuring the dominant polynomial coefficient-ranging and monotonicity-reduced Bernstein paths in a way that is compatible with the prepared/many-box API and future numeric batch buffers. Do not optimize the old single-box object path in a way that would have to be discarded for batch/GPU execution.

In parallel, begin assembling the external engineering corpus and platform CI so that correctness/usability findings can influence the remaining language and domain work before the API is frozen.

21 Final release perspective

The core research question that motivated AutoIA has largely moved from "can this architecture be made rigorous?" to "can it be made broad, fast, and predictable enough for engineers outside the development effort?" The major mathematical pieces are now present. The remaining Version-1 work should therefore emphasize hardening, corpus-driven supported-language decisions, performance where profiling shows real bottlenecks, repeated-query scalability, and clear failure semantics.

A disciplined Version-1 scope is itself important research infrastructure. It creates a stable platform on which more ambitious semantic primitives, GPU proof kernels, richer generator structures, and non-box uncertainty can be evaluated without continually changing the foundational guarantee.